NOIP 模拟赛 2

题目名称	公交线路	连通子集	最小表示串
输入文件名	bus.in	tree.in	string.in
输出文件名	bus.out	tree.out	string.out
每个测试点时限	1 sec	1 sec	2 sec
内存限制	128M	128M	128M
测试点数目	10	10	10
每个测试点分值	10	10	10
是否有部分分	无	无	无
题目类型	传统	传统	传统

提交源程序须加后缀

对于 Pascal 语言	bus.pas	tree.pas	string.pas
对于 C　　语言	bus.c	tree.c	string.c
对于 C++　语言	bus.cpp	tree.cpp	string.cpp

视现场评测机情况，题目的时间限制及空间限制均可能发生调整。

公交线路

【问题描述】

A城的地图可以被看作一个大小为$n \times m$的矩形。城中有n条东西向道路和m条南北向道路。第i条东西向道路和第j条南北向道路交汇于路口(i, j)。

每一条道路中都运行有一条公交线路。这条公交线路可以将乘客从道路上的某一个路口运送到另一个路口。乘坐公交需要购买车票，不同公交线路的车票售价可能会有所不同。

最近，A城的公交系统中产生了q次事件。每一起事件可能为如下类型：

$0\ i\ c$：第i条东西向道路上公交线路的车票价格修改为c。

$1\ j\ c$：第j条南北向道路上公交线路的车票价格修改为c。

$2\ x_1\ y_1\ x_2\ y_2$：有一名乘客希望从路口(x_1, y_1)出发，去往路口(x_2, y_2)。他希望知道他需要花费的车票价格之和最少为多少。你需要对他的询问作出回答。

【数据规模和约定】

对于100%的数据，满足$1 \leq n, m \leq 100000$，车票价格不会超过100000。

【简要题解】

两个路口存在某一维坐标相同等特殊情况可以特判考虑。考虑一般情况：两个路口的两维坐标均不相同。在这种情况下，两个路口之间通行的方法至多有如下四种情况可能成为最优解：

1、先走x_1线再走y_2线

2、先走y_1线再走x_2线

3、走x_1、x_2线以及y方向上花费最小的一条线

4、走y_1、y_2线以及x方向上花费最小的一条线

前两种情况可以直接计算权值，后两种情况则需要获取两个方向上的公交线路的最小值。该问题可以规约成维护一个可重集合的最小值，支持加入、删除某个元素。使用数据结构（堆、平衡树等）可以很方便地维护该信息，使用离线算法则可以不使用数据结构地维护该信息。

连通子集

【问题描述】

对于无向连通图$G(V,E)$，它的一个连通子集为V的一个子集，并满足子集中任意两点之间存在一条道路只包含在子集中的点。

现在给定一张n个点$n-1$条边的无向连通图G，每个节点有两个点权a和b。对于图G的一个连通子集，它的权值为子集中所有点的a权值之和与b权值之和的乘积。

对于图G的所有不同的连通子集，你需要求出它们的权值之和。由于答案可能很大，你只需要输出答案对10^9+7取模后的值。

【数据规模和约定】

对于100%的数据，满足$1 \le n \le 800000$，a权值和b权值均为$\le 10^9$的正整数。

【简要题解】

使用树DP解决本题：任取一个节点作为根，对于每个连通子集，在这个连通子集离根最近的点上考虑该连通子集。称这个点为连通子集的根。

对于一个节点，记录如下四个信息：

res：以当前点为根的所有连通子集的权值和

$suma$：以当前点为根的所有连通子集的a权值和

$sumb$：以当前点为根的所有连通子集的权值和

cnt：以当前点为根的所有连通子集的个数

假设树DP进行到点u，考虑将点u的某个儿子v的信息更新进当前点的信息中，转移方程如下：

$u.res = u.res \times (v.cnt+1) + u.cnt \times v.res + u.suma \times v.sumb + u.sumb \times v.suma$

$u.suma = u.suma \times (v.cnt+1) + u.cnt \times v.suma$

$u.sumb = u.sumb \times (v.cnt+1) + u.cnt \times v.sumb$

$u.cnt = u.cnt \times (v.cnt+1)$

其中，$v.cnt$需要$+1$是因为需要考虑儿子v完全不取的情况。

按照上述方程进行转移，并统计每个节点的res信息的和即可。

最小表示串

【问题描述】

对于一个十进制下的正整数，它可以被看作一个数，也可以被看作一个字符串。当我们把它看作一个数时，我们可以研究它在数字上的性质。比如，我们可以计算两个正整数的差，来判断两个数字之间的大小关系。当我们把它看作一个字符串时，我们可以研究它在字符串的性质。比如，我们可以判断一个数字串是否是最小表示串。最小表示串的定义是：如果将一个字符串的开头的若干位移动到字符串的末尾，所得到的字符串的字典序均不小于原字符串的字典序，则认为一个字符串是最小表示串。

现在，给定一个十进制下的正整数，我们需要同时研究它关于数的性质以及关于字符串的性质。你需要求出：所有小于给定正整数且满足最小表示串定义的字符串中，最大的是哪一个？

【数据规模和约定】

对于100%的数据，满足$1 \leq n \leq 30000$，$1 \leq T \leq 30$。

【简要题解】

考虑答案字符串的构成。答案字符串的前若干位会和输入的字符串相同，接着有某一位存在不同。可以发现，第一个不同的位比输入字符串的对应位小1显然是最优的。对于不同位之后的字符，使用字符$'9'$进行填充一定是最优的。

如此我们可以得到$O(n)$个答案的候选串，用$O(n)$的最小表示串判别算法进行判别即可得到一个$O(n^2)$的算法。

接下来，我们需要进一步考虑$O(n)$个候选串中真正可行的有哪些。假设我们需要把位置i所在的字符减去1，并且把之后的字符都修改为$'9'$。通过KMP算法，我们可以求出以位置i结尾的输入串的前缀的后缀中，和原串匹配的最长长度是多少。

如果匹配长度不为0，则位置i减去1后的字符串很显然包含一个比原串小的中间子串，这个位置不可行。

如果匹配长度为0，这说明位置i出现了失配。根据位置$i-1$对应的$next$数组的取值，我们可以得到位置i的失配位置j。

如果位置i的字符小于位置j的字符，这说明我们直接得到了一个比原串小的中间子串。这同时表明位置i之后的位置都一定不能成为候选。

如果位置i的字符大于位置j的字符，这说明这一位置可能成为候选。

在绝大部分情况下，位置i都是可行的。只有一种特殊情况需要我们进行额外判断：位置i的字符恰好比位置j的字符大1，并且位置j后接了许多字符$'9'$。如果后接的$'9'$的个数比位置i之后要填充的$'9'$的个数多，则位置i不可行；如果后接的$'9'$的个数较少，则位置i可行；如果相等，则仅当位置j与后接$'9'$构成的子串恰好为原串长度的一半时，位置i可行，否则位置i不可行。根据上述判别算法即可求出最靠后的可行的位置i。